FieldTrack

A Complete Engineering Curriculum for the Field Sales App

Architecture, folder structure, core JavaScript/React/Express/PostgreSQL concepts, and full end-to-end user-flow traces through the real, current codebase — written as a self-study reference for becoming this application's primary engineer.

Compiled 2026-07-30 · Reference document, not a substitute for reading the source directly

TABLE OF CONTENTS

Part I — Architecture & Mental Model

1.1 What this application is, in one paragraph

1.2 The three programs and who uses each one

1.3 Architecture diagram

1.4 Data ownership — who creates, validates, and stores each kind of data

1.5 Why the backend is the only thing allowed to touch the database

Part II — Folder Structure

2.1 mobile/ — full tree and rationale

2.2 backend/ — full tree and rationale

2.3 web/ — full tree and rationale

2.4 Cross-folder communication diagram

Part III — Foundations Glossary

3.1 Functions and arrow functions

3.2 Destructuring and the spread operator

3.3 Promises

3.4 async/await

3.5 JSX and React components

3.6 React hooks: useState, useEffect, useContext, useRef

3.7 Express middleware and the request/response cycle

3.8 JWT authentication

3.9 SQL joins and parameterized queries

Part IV — User Flow Traces

4.1 App boot & splash flow

4.2 Login flow, end to end

4.3 Day Check-In flow

4.4 Dealer Check-In flow (geofencing)

4.5 Offline queue & sync flow

Part V — Key File Deep Dives

5.1 mobile/App.js

5.2 mobile/screens/LoginScreen.js

5.3 mobile/src/services/api.js

5.4 mobile/src/services/syncManager.js

5.5 mobile/src/services/location.js

5.6 backend/src/index.js

5.7 backend/src/routes/auth.routes.js

5.8 backend/src/middleware/auth.middleware.js

5.9 backend/src/routes/attendance.routes.js

5.10 backend/src/routes/visits.routes.js

5.11 backend/src/utils/haversine.js and businessDay.js

5.12 backend/src/db/schema.sql

Part VI — Closing

6.1 Common mistakes across this codebase's domain

6.2 Interview questions bank

6.3 Exercises and coding challenges

6.4 What to read next / study roadmap

PART I

Architecture & Mental Model

1.1 What this application is, in one paragraph

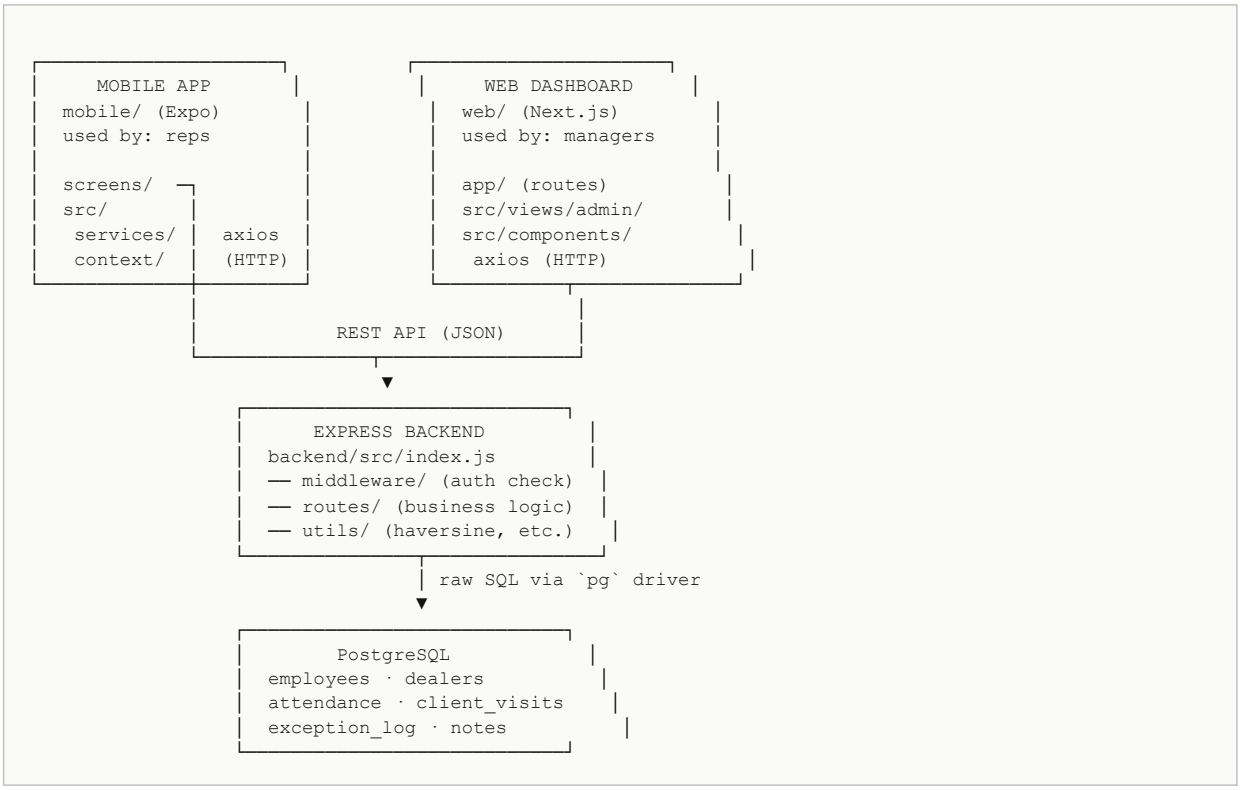
FieldTrack (the working name used inside `backend/package.json` as `fieldtrack-backend`) is a field-sales tracking system. A company employs sales representatives ("reps") who travel between dealer locations (shops, distributors) during the workday. The system answers three questions a manager cares about: did a rep start and end their workday (attendance), did they actually visit the dealers they claim to have visited (geofenced check-ins), and where are the gaps or suspicious patterns (exceptions, out-of-radius events, interrupted visits)? Everything else in the codebase — GPS capture, offline queuing, JWT authentication, the manager's heatmap — exists in service of answering those three questions reliably, even when a rep's phone has no signal for hours at a time.

1.2 The three programs and who uses each one

The repository root contains three independent applications. They are not a single monorepo build — each has its own `package.json`, its own `node_modules`, and is started with its own command. They share nothing directly; the only thing that connects them is the backend's HTTP API.

Folder	Technology	Used by	Purpose
`mobile/`	Expo (React Native 0.81, React 19)	Sales reps, in the field	Day check-in/out, dealer check-in/out, notes, history
`backend/`	Express (Node.js) + PostgreSQL via raw `pg` driver	Nobody directly — it's the API both apps talk to	Business logic, the only thing allowed to read/write the database
`web/`	Next.js 15 (React 19)	Managers, in the office	Dealer/employee management, heatmap, reports

1.3 Architecture diagram



The single most important rule this diagram encodes: **neither the mobile app nor the web dashboard is ever allowed to query the database directly.** Both only ever send HTTP requests to the Express backend, and the backend is the only process in the entire system holding a database connection (see `backend/src/db/pool.js`, Section 5.12 companion). This matters for a concrete reason: a phone can be rooted, its network traffic intercepted, its local storage read. If the mobile app could query Postgres directly using embedded credentials, those credentials would be extractable from the app binary. Routing everything through the backend means the only credentials that exist are a per-employee JWT that expires in 8 hours and carries no direct database access — inspecting or replaying it lets an attacker impersonate that one employee at worst, not read or write the entire database.

1.4 Data ownership — who creates, validates, and stores each kind of data

Data	Created by	Validated by	Stored in
Username / password	Rep or manager types it into `LoginScreen.js`	Backend (`auth.routes.js`: `bcrypt.compare`)	`employees.password_hash` (bcrypt hash, never plaintext)
GPS coordinates at check-in	Phone hardware, read via `expo-location` in `location.js`	Backend (`visits.routes.js`: Haversine distance vs. dealer radius)	`client_visits.check_in_lat/lng`
Dealer location & geofence radius	Manager, via the web dashboard's dealer form	Backend route validation (numeric ranges)	`dealers.latitude/longitude/radius_meters`
Access token (JWT)	Backend, at successful login (`jwt.sign`)	Backend, on every subsequent request (`jwt.verify` in `auth.middleware.js`)	Phone's `expo-secure-store` (never the database)
Offline-queued actions	Mobile app, when a request fails with a network error	Backend, when the queued action finally reaches it later	Phone's `AsyncStorage` under key `@offline_action_queue`

> **Why this table matters for interviews and for debugging:** almost every "weird bug" in a system like this comes down to a violation of one of these rows — e.g. trusting a GPS coordinate the client computed instead of re-deriving distance server-side, or storing a token somewhere an attacker's JS could read it (this codebase deliberately uses `expo-secure-store`, which is backed by the OS keychain/keystore, specifically to avoid that).

1.5 Why the backend is the only thing allowed to touch the database

Think of the backend as a bank teller window. Customers (the mobile app, the web dashboard) never walk behind the counter into the vault (PostgreSQL) themselves — they hand a request to the teller, and the teller decides whether to honor it, based on rules the customer cannot override (is this employee active? does this JWT still verify? does this rep own this attendance record?). If customers could reach into the vault directly, every rule enforcement would depend on the client behaving honestly — and a modified/rooted phone, or a browser's dev tools on the web dashboard, can never be trusted to behave honestly. This is why, throughout Part IV and Part V, you will see the same pattern repeated in nearly every route: re-verify ownership (`WHERE employee_id = $1`), re-derive anything security-relevant server-side (distance via Haversine, not a client-supplied "inside radius" boolean), and only then write to the database.

PART II

Folder Structure

2.1 mobile/ — full tree and rationale

```
mobile/
├── App.js                    entry component — owns ALL top-level state
├── index.js                  Expo bootstrap (registers App.js with the OS)
├── app.json / eas.json       Expo build configuration
├── screens/                  one file per full-screen view (flat, not nested)
│   ├── SplashScreen.js
│   ├── LoginScreen.js
│   ├── HomeScreen.js
│   ├── DayCheckInScreen.js / DayCheckOutScreen.js
│   ├── DealerDirectoryScreen.js
│   ├── DealerCheckInScreen.js / DealerCheckOutScreen.js
│   ├── HistoryScreen.js
│   ├── NotesScreen.js / NoteEditorScreen.js
│   └── ProfileScreen.js
├── src/
│   ├── components/            small reusable UI pieces, grouped by kind
│   │   ├── buttons/  cards/  headers/  inputs/  loaders/
│   ├── context/
│   │   └── AppStateContext.js  shares App.js's state with tab screens
│   ├── navigation/
│   │   └── MainTabs.js         the bottom-tab shell (Home/Dealers/History/Profile)
│   ├── services/               all "talk to the outside world" logic
│   │   ├── api.js              axios instance + token attach/refresh
│   │   ├── location.js         GPS acquisition + reverse geocoding
│   │   ├── syncManager.js      offline action queue + auto-flush
│   │   └── visitMonitor.js     periodic in-visit GPS pings
│   ├── theme/                  colors, spacing, typography constants
│   └── utils/                   small pure helper functions
├── assets/ , app-icon/           images, icons
└── .expo/                        Expo's local dev cache (gitignored)
```

Why `screens/` lives at the top level, not under `src/`: this is a convention choice, not a technical requirement — React Native doesn't care where files live. This codebase keeps "whole-page" components at the root and reserves `src/` for everything that supports those pages (services, shared components, theme, context). A new file that renders a full screen the user navigates to goes in `screens/`; a new file that's a helper reused by several screens goes in `src/`.

What should never go in `screens/`: direct `fetch` / `axios` calls without going through `src/services/api.js`'s shared instance (you'd lose the automatic token-attach and refresh-on-401 interceptor logic described in Section 5.3), and business rules that belong on the server (e.g. a screen should never itself decide "is this rep close enough to the dealer" — it sends coordinates and the backend decides, per Section 1.5).

What should never go in `src/services/`: JSX / UI rendering. A service file's job is data and side effects (network calls, storage reads/writes) — if a file in `services/` starts importing `react-native` components to render something, that's a sign the UI logic drifted into the wrong layer.

2.2 backend/ — full tree and rationale

```
backend/
├── src/
│   ├── index.js                  entry point — wires up Express, middleware, routes
│   ├── db/
│   │   ├── pool.js                 the ONE shared PostgreSQL connection pool
│   │   ├── schema.sql              full table definitions (raw SQL, no ORM)
│   │   ├── migrate.js              runs schema.sql against the configured DB
│   │   └── seed.js                 inserts demo/test data
│   ├── middleware/
│   │   └── auth.middleware.js      requireAuth (verify JWT) / requireRole (RBAC)
│   ├── routes/                     one file per resource — business logic lives HERE
│   │   ├── auth.routes.js
│   │   ├── attendance.routes.js
│   │   ├── visits.routes.js
│   │   ├── dealers.routes.js
│   │   ├── employees.routes.js
│   │   ├── dashboard.routes.js
│   │   ├── reports.routes.js
│   │   ├── geocode.routes.js
│   │   └── notes.routes.js
│   └── utils/                      small stateless helpers shared by routes
│       ├── haversine.js            GPS distance calculation
│       ├── businessDay.js          "day rolls over at 5am, not midnight" logic
│       ├── activityLog.js          human-readable audit log lines
│       └── logger.js               winston logger configuration
├── tests/                          jest + supertest, mirrors src/ structure
├── logs/                           winston's rotated log files (gitignored)
├── .env / .env.example              secrets (gitignored) / template (committed)
└── package.json                    fieldtrack-backend
```

Why there's no `controllers/` or `models/` folder: many Express tutorials split "controller" (request handling) from "model" (database access) into separate files. This codebase deliberately keeps them together, one file per resource in `routes/` — a route handler both parses the request AND runs its SQL query inline. The backend README explains the reasoning: with a small, well-understood schema (4-6 tables) and no ORM, the extra indirection of separate model files mostly adds navigation overhead without buying real separation, since there's no swappable data layer to isolate the routes from. This is a deliberate, justified simplification for this project's size — not an oversight. If the schema grew to dozens of tables with reused query logic across many routes, extracting a data-access layer would become worth it again.

What should never go in `routes/`: raw `console.log` for anything other than throwaway local debugging — the project already has a real logger (`utils/logger.js`, backed by winston) and every route in the codebase logs errors through it (`logger.error(...)`), so a stray `console.log` would be inconsistent with the rest of the file and invisible to the rotated log files in production. Also: no direct `process.env` reads for values that have defaults/validation elsewhere — e.g. `businessDay.js` centralizes the "day boundary hour" logic specifically so it isn't reimplemented slightly differently in each route that needs it.

2.3 web/ — full tree and rationale

```
web/
├── app/                      Next.js "App Router" — file-based routing
│   ├── layout.jsx              wraps every page (fonts, providers, global shell)
│   └── page.jsx                the dashboard's main page
├── src/
│   ├── views/admin/            top-level tab views assembled from components
│   │   ├── DealersTab.jsx
│   │   └── EmployeesTab.jsx
│   ├── components/              grouped by kind, same philosophy as mobile/
│   │   ├── buttons/ cards/ dialogs/ filters/ forms/ headers/ tables/
│   │   └── heatmap/               deck.gl + Google Maps heatmap layer
│   ├── hooks/                    custom React hooks shared across views
│   └── theme/
├── public/                       static assets served as-is
├── next.config.mjs
└── package.json                  next@15, react@19, @react-google-maps/api, deck.gl
```

Note on the stale README: `web/README.md` still describes a "React + Vite" project. The actual `package.json` scripts (`next dev`, `next build`, `next start`) confirm the project was migrated to Next.js at some point and the README was never updated — this was also the reason `web/dist/` (Vite's build-output folder name) was found sitting in the repo as dead weight during the cleanup pass that produced this document; Next.js outputs to `.next/` instead, never `dist/`.

2.4 Cross-folder communication diagram

```
mobile/src/services/api.js ─┐
                            │  POST /api/auth/login
                            │  GET  /api/attendance/today
web/src/... (axios calls) ──┤  POST /api/visits/check-in
                            │  GET  /api/dashboard/...
                            ▼
                  backend/src/index.js
                  mounts each routes/*.js file at a URL prefix:
                    /api/auth       → auth.routes.js       (public)
                    /api/attendance → attendance.routes.js  (requireAuth)
                    /api/visits     → visits.routes.js       (requireAuth)
                    /api/dealers    → dealers.routes.js       (requireAuth)
                    /api/dashboard  → dashboard.routes.js      (requireAuth + requireRole('manager'))
                    /api/employees  → employees.routes.js      (requireAuth + requireRole('manager'))
                            │
                            ▼
                  backend/src/db/pool.js  →  PostgreSQL
```

Nothing in `mobile/` ever imports anything from `backend/` or `web/`, and vice versa. The *only* shared contract between all three folders is the shape of the JSON each API endpoint accepts and returns — which is why Part IV traces those exact JSON shapes at each hop.

PART III

Foundations Glossary

This part is a condensed reference for concepts that recur constantly in Part IV and Part V. If a concept below is already solid for you, skim it — it's written so each entry stands alone rather than requiring you to read the others first.

3.1 Functions and arrow functions

A function is a reusable recipe: defining it does nothing by itself; calling it is what actually runs the steps. Arrow functions (`const f = (x) => ...`) are today's default syntax across this whole codebase. The single rule to hold onto: **no curly braces after `=>` means the one expression's value is automatically returned; curly braces mean you must write `return` yourself, or the function silently gives back `undefined`.**

```
const double = (n) => n * 2;              // implicit return — returns 10 for double(5)
const double2 = (n) => { return n * 2; };  // explicit return — same result
const broken  = (n) => { n * 2; };         // braces, no return — ALWAYS undefined
```

3.2 Destructuring and the spread operator

Destructuring unpacks named fields out of an object (or positional items out of an array) into their own variables, without changing the original. It shows up in nearly every function signature in this codebase:

```
// backend/src/routes/auth.routes.js
const { username, password } = req.body;
// mobile/screens/LoginScreen.js
const { accessToken, refreshToken, employee } = response.data;
```

The spread operator (`...`) does the opposite conceptually — it expands an object/array's contents into a new one, used constantly for "update state without mutating the old state" in React:

```
// mobile/App.js — appending a new visit without mutating the old visits array
setVisits((prev) => [...prev, newVisit]);
// mobile/src/services/api.js — copying an action's data before rewriting one field
const data = { ...action.data };
```

> **Why not just mutate the array/object directly?** React decides whether to re-render a component partly by comparing whether state *changed identity* (a new object/array), not just whether its contents changed. Mutating the existing array in place and calling `setVisits(visits)` with the same reference can cause React to think nothing changed and skip the re-render — a very common real bug class in React apps.

3.3 Promises

A Promise represents a value that doesn't exist yet but will (or will fail to) exist later — like a restaurant buzzer handed to you after ordering. The code immediately after a Promise-returning call does NOT wait for it; JS keeps running, and the promise's `.then()` / `.catch()` (or, more commonly in this codebase, an `await` inside a `try/catch` — see 3.4) run later, whenever the operation actually settles. **Critically: an unhandled/rejected promise never retries itself. Retrying is something a developer must code explicitly** — this codebase does exactly that in `syncManager.js`'s `MAX_RETRIES = 8` bounded-retry logic (Section 5.4).

3.4 async/await

`async/await` is syntax sugar over promises — it lets asynchronous code read top-to-bottom like synchronous code, without changing what actually happens underneath (execution still yields control back to the rest of the program while waiting). Marking a function `async` makes it always return a Promise; `await` inside it pauses that function (not the whole program) until the awaited promise settles.

```
// mobile/screens/LoginScreen.js — the actual pattern used everywhere in this codebase
const handleLogin = async () => {
  setLoading(true);
  try {
    const response = await api.post('/auth/login', { username, password });
    const { accessToken, refreshToken, employee } = response.data;
    await SecureStore.setItemAsync('accessToken', accessToken);
    // ...
  } catch (error) {
    // any rejection anywhere above this line lands here
    Alert.alert('Login Failed', error.response?.data?.error || 'Could not connect to the server.');
  } finally {
    setLoading(false); // runs whether login succeeded or failed
  }
};
```

What breaks if `try/catch` is removed here: if `api.post` rejects (wrong password → backend returns 401 → axios throws) and there is no surrounding `try/catch`, the rejection becomes an unhandled promise rejection inside an event handler — the loading spinner would likely get stuck on forever, since `setLoading(false)` in the `finally` block would never run.

3.5 JSX and React components

JSX is the HTML-looking syntax inside `.js`/`.jsx` files — it is not actually HTML; it's syntax that compiles down to plain JavaScript function calls (`React.createElement(...)`) that build a tree describing what should appear on screen. A React *component* is just a JavaScript function that returns JSX. Every screen in `mobile/screens/` and every file in `web/src/components/` is exactly this shape:

```
export default function LoginScreen({ onLoginSuccess }) {
  // ...state and functions...
  return (
    <KeyboardAvoidingView style={styles.container}>
      <ScrollView>
        <TextField label="Username" value={username} onChangeText={setUsername} />
      </ScrollView>
    </KeyboardAvoidingView>
  );
}
```

`{ onLoginSuccess }` in the function signature is destructuring (3.2) applied to *props* — the single object React passes every component containing whatever attributes its parent gave it in JSX (`<LoginScreen onLoginSuccess={...} />`). `value={username}` and `onChangeText={setUsername}` are exactly this — props being passed down.

3.6 React hooks: useState, useEffect, useContext, useRef

useState gives a component memory that survives across re-renders and triggers a re-render when changed:

```
const [username, setUsername] = useState('');  // LoginScreen.js:20
// calling setUsername('sanjay') schedules a re-render with the new value
```

useEffect runs side effects (things that reach outside pure rendering: timers, subscriptions, network calls) after a render, and optionally cleans them up. Its second argument (the "dependency array") controls when it re-runs — `[]` means "once, on mount"; `[employee]` means "on mount, and again any time `employee` changes":

```
// App.js:86-90 — starts auto-sync only while a session is active
useEffect(() => {
  if (!employee) return;
  startAutoSync();
  return () => stopAutoSync();   // cleanup: runs before the next effect, or on unmount
}, [employee]);
```

Why the cleanup function matters here: without `return () => stopAutoSync()`, logging out and back in as a different employee would leave the previous session's NetInfo listener still running alongside a new one — a leak that would eventually cause duplicate sync attempts.

useContext reads a value provided further up the component tree without it being threaded through every intermediate component as a prop ("prop drilling"). This app defines exactly one: `AppStateContext` (Section 5.1 companion) — `App.js` provides employee/attendance/visits and their handlers once, and any nested tab screen calls `useAppState()` to read them directly.

useRef holds a mutable value that does NOT trigger a re-render when changed, and persists across renders — used in `LoginScreen.js` to hold a timer id (`coldStartTimerRef`) so it can be cleared later without needing to be part of render-triggering state.

3.7 Express middleware and the request/response cycle

Express processes every incoming HTTP request through an ordered chain of functions called middleware, each of which can inspect/modify the request, end the response early, or call `next()` to pass control along. `backend/src/index.js` is a textbook example of this chain, top to bottom: `helmet()` (security headers) → `cors(...)` (origin checks) → `express.json()` (parses the request body) → rate limiters → the route handlers themselves → a 404 catch-all → a global error handler. `requireAuth` and `requireRole('manager')` (Section 5.8) are also middleware — they are inserted between the URL and the route handler specifically to reject unauthorized requests before any route code (and therefore any database query) ever runs.

```
// backend/src/index.js:154
app.use('/api/dashboard', requireAuth, requireRole('manager'), dashboardRouter);
// reading left to right: for any /api/dashboard/* URL, first run requireAuth,
// then (only if that called next()) run requireRole('manager'),
// then (only if that also called next()) hand off to dashboardRouter's own routes
```

3.8 JWT authentication

A JSON Web Token (JWT) is a signed, tamper-evident string the server hands the client after a successful login, and the client sends back on every subsequent request (in this app: the `Authorization: Bearer <token>` header, attached automatically by an axios interceptor in `api.js`). "Signed" means the server can verify the token wasn't altered without needing to store anything about it server-side (no session table) — it re-computes the signature using a secret (`process.env.JWT_SECRET`) and checks it matches. This codebase issues two tokens: a short-lived **access token** (8h, used on every request) and a longer-lived **refresh token** (7d, used only to obtain a new access token once the old one expires) — see Section 5.7 and 5.3 for exactly how this pair is used together.

> **Why two tokens instead of one long-lived one:** if a single 7-day token were sent on every request, stealing it (e.g. from an intercepted request) grants an attacker a full week of access. Splitting it means the token sent constantly (access) is only valid for 8 hours, while the token that could grant a longer window (refresh) is sent far less often (only at refresh time), narrowing the exposure window for the credential that matters more.

3.9 SQL joins and parameterized queries

A **JOIN** combines rows from two tables based on a matching column — e.g. `visits.routes.js`'s list endpoint joins `client_visits` to `dealers` and `employees` so one query can return a visit alongside the dealer's name and the employee's name, instead of three separate round trips:

```
SELECT cv.id, e.name AS employee_name, d.name AS dealer_name, cv.check_in_time
FROM client_visits cv
JOIN attendance a ON a.id = cv.attendance_id
JOIN dealers d    ON d.id = cv.dealer_id
JOIN employees e  ON e.id = a.employee_id
WHERE cv.dealer_id = $1
```

A **parameterized query** is what `$1` , `$2` , etc. represent above — placeholders the `pg` driver fills in safely, separately from the query text itself, rather than the code string-concatenating user input directly into the SQL string. Every single query in this backend uses this pattern.

> **What breaks without parameterization — SQL injection:** if `auth.routes.js` instead built its query as `` `SELECT * FROM employees WHERE username = '${username}'` `` , a malicious username like `' OR '1'='1` would change the query's actual meaning and could bypass the password check entirely. Parameterized queries make this class of attack structurally impossible, because the value is never interpreted as part of the SQL syntax.

PART IV

User Flow Traces

4.1 App boot & splash flow

```
mobile/index.js
    │  registerRootComponent(App)
    ▼
mobile/App.js  →  renders <Stack.Navigator> starting at "Splash"
    ▼
SplashRoute (App.js:33-57)
    │  useEffect fires once, waits 1500ms (minimum splash display time)
    │  reads SecureStore 'accessToken' + AsyncStorage 'employeeData'
    ├─ both present  → setEmployee(...) → fetchTodayState() → navigate to "MainTabs"
    └─ either missing → navigate to "Login"
```

Walking this in order: `index.js` is the very first file executed — its only job is to hand `App` to `registerRootComponent`, which is Expo/React Native's way of saying "this is the root of the UI tree; mount it." Inside `App.js`, the top-level `Stack.Navigator` (a React Navigation concept — a stack of full-screen routes you can push/pop/replace) starts at the "Splash" route. The `SplashRoute` function is a small wrapper (not a full screen file) whose entire purpose is to run one `useEffect` on mount: wait 1.5 seconds (so the splash screen doesn't flash instantly even on a fast device — a deliberate UX choice, not a technical necessity), then check whether a previous session's token still exists in secure storage. If yes, the session is restored without re-login; if no, the user lands on `LoginScreen`.

> **Why check both `accessToken` AND `employeeData` instead of just the token:** the token proves the session is valid to the server, but `employeeData` (name, role, region) is what the UI needs to render immediately without waiting on a network round trip — storing both avoids a blank/loading flash for basic profile info the app already has locally.

4.2 Login flow, end to end

```
LoginScreen.js: user types username/password, taps "Log in"
    ▼
handleLogin() (LoginScreen.js:28)
    │  setLoading(true)
    │  await api.post('/auth/login', { username, password })
    ▼
mobile/src/services/api.js
    │  request interceptor attaches nothing yet (no token exists pre-login)
    │  axios sends: POST http:///api/auth/login
    │  body: { "username": "...", "password": "..." }
    ▼
                   ── network ──
    ▼
backend/src/index.js
    │  app.use('/api/auth', authRouter)   ← PUBLIC route, no requireAuth
    │  loginLimiter middleware: max 20 attempts / 15 min per IP
    ▼
backend/src/routes/auth.routes.js  →  router.post('/login', ...)
    │  destructure { username, password } from req.body
    │  400 if either is missing
    │  SELECT ... FROM employees WHERE LOWER(username) = LOWER($1)
    │  401 'Username not found' if no match or is_active = false
    │  bcrypt.compare(password, employee.password_hash)
    │  401 'Incorrect password' if it fails
    │  signAccessToken(employee)   → jwt.sign(..., expiresIn: '8h')
    │  signRefreshToken(employee)  → jwt.sign(..., expiresIn: '7d')
    ▼
    200 response: { accessToken, refreshToken, employee: {id,name,username,role,region} }
    ▼
                   ── network ──
    ▼
LoginScreen.js (back in handleLogin, after await resolves)
    │  destructure { accessToken, refreshToken, employee } from response.data
    │  SecureStore.setItemAsync('accessToken', accessToken)
    │  SecureStore.setItemAsync('refreshToken', refreshToken)
    │  AsyncStorage.setItem('employeeData', JSON.stringify(employee))
    │  onLoginSuccess()  ← prop passed in from App.js's <Stack.Screen name="Login">
    ▼
App.js: handleLoginSuccess() (App.js:194)
    │  re-reads AsyncStorage 'employeeData', setEmployee(parsed)
    │  await fetchTodayState()  → GET /attendance/today (see 4.3)
    ▼
navigation.replace('MainTabs')  ← user now sees the Home tab
```

Every numbered concept from Part III appears in this one flow: `async/await` throughout, destructuring at nearly every step, a parameterized SQL query, bcrypt password comparison, and JWT signing. Notice specifically *why* the backend responds with a distinct `'Username not found'` vs. `'Incorrect password'` message (auth.routes.js:56, 61) — the code comment explains this is a deliberate choice for this specific app: it's an internal tool with a small, known set of accounts, so the usual security concern about "username enumeration" (an attacker learning which usernames exist) doesn't apply here, and the specific message helps a rep catch their own typo instead of assuming the server is broken. This would be the wrong choice for a public-facing consumer app with millions of accounts — context changes what's "correct."

4.3 Day Check-In flow

```
DayCheckInScreen: rep taps "Check In"
    ▼
location.js: getCurrentLocation()
    │  Location.requestForegroundPermissionsAsync()
    │  loop up to 3 attempts, keep the most accurate reading (≤20m accuracy = "good enough")
    │  15s timeout per attempt (withTimeout wrapper) so a bad GPS fix can't hang forever
    ▼
api.post('/attendance/check-in', { lat, lng })
    ▼
backend/src/routes/attendance.routes.js  →  router.post('/check-in', ...)
    │  parseCoord() validates lat ∈ [-90,90], lng ∈ [-180,180]
    │  INSERT INTO attendance (employee_id, business_date, check_in_time, check_in_lat, check_in_lng, ...)
    │  business_date computed via businessDay.js's businessDateExpr('NOW()')
    │  ON CONFLICT (employee_id, business_date) DO NOTHING   ← atomic "already checked in" guard
    ├─ 0 rows returned → 409 'Already checked in today' (re-fetches the existing id)
    └─ 1 row returned  → 201 { attendance: {id, check_in_time, check_in_lat, check_in_lng} }
    ▼
DayCheckInScreen: onCheckIn(data) → App.js's handleDayCheckIn(newAttendance) → setAttendance(...)
navigation.navigate('MainTabs')
```

> **Why `ON CONFLICT ... DO NOTHING` instead of "check if already checked in, then insert":** a check-then-insert approach has a race condition — if a rep double-taps the button, or the app retries a request that timed out but actually succeeded, two near-simultaneous requests could both pass the "not checked in yet" check before either has inserted its row, resulting in two attendance rows for the same day. The unique index on `(employee_id, business_date)` (schema.sql, Section 5.12) combined with `ON CONFLICT DO NOTHING` makes this atomic at the database level — the database itself guarantees only one row can ever exist for that pair, regardless of how many requests arrive concurrently.

The "business day" concept (`businessDay.js`, Section 5.11) is worth pausing on: a rep who checks in at 11pm and works past midnight is still considered "in the same business day's session" until 5am (configurable via `DAY_BOUNDARY_HOUR`), not until calendar midnight. Without this, a rep working a late shift past midnight would appear to the system as needing a second, fresh day-check-in immediately after midnight, and any "already checked in today" guard would fail to catch a genuine duplicate.

4.4 Dealer Check-In flow (geofencing)

```
DealerCheckInScreen: rep selects a dealer, taps "Check In"
    ▼
location.js: getCurrentLocation()  (same acquisition logic as 4.3)
    │  if accuracyMeters > MAX_ACCEPTABLE_ACCURACY_METERS (30m) → reject client-side, ask rep to retry
    ▼
api.post('/visits/check-in', { attendance_id, dealer_id, lat, lng, accuracy_meters, reason? })
    ▼
backend/src/routes/visits.routes.js  →  router.post('/check-in', ...)
    │  re-validates accuracy server-side too (never trust the client-side check alone)
    │  SELECT attendance WHERE id = $1 AND employee_id = $2   ← ownership check
    │  SELECT dealer WHERE id = $1                             ← get registered lat/lng/radius
    │  haversineKm(dealer.lat, dealer.lng, rep.lat, rep.lng) * 1000  → distanceM
    │  insideRadius = distanceM <= (dealer.radius_meters ?? 200)
    ├─ outside radius AND reason.length < 20 chars → 422 'reason_required'
    │      (client must re-submit with a written justification)
    └─ otherwise → proceed:
    │  haversineKm(previous location, this location) → distFromPrev
    │      ("previous location" = last check-out, or today's day check-in if none)
    │  INSERT INTO client_visits (..., check_in_inside_radius, justification_note, ...)
    │  UPDATE attendance SET total_distance_km = total_distance_km + distFromPrev
    │  if outside radius → INSERT INTO exception_log (for manager review)
    ▼
    201 { visit: { id, dealer_id, check_in_time, check_in_distance_m, check_in_inside_radius, ... } }
```

This is the flow where Section 1.5's principle is most visible: the mobile app never tells the server "I am inside the radius" — it only ever sends raw coordinates and a numeric accuracy figure, and the server independently recomputes the distance and decides. A modified client claiming `insideRadius: true` in its request body would have zero effect, because that field doesn't even exist in what the server reads from `req.body` — the server computes its own `insideRadius` variable from scratch using Haversine (Section 5.11) every single time.

> **Why check GPS accuracy at all, on both client and server:** a GPS reading with 50+ meters of error (common indoors, in urban canyons, or right after the GPS radio wakes from sleep) could falsely show a rep as outside a 200m dealer radius when they're actually standing inside it, or vice versa. Both layers reject a reading worse than the threshold rather than silently trusting an unreliable fix — the client check saves a wasted round trip for an obviously bad reading, the server check is the one that actually can't be bypassed.

4.5 Offline queue & sync flow

```
Rep taps "Check In" with no network connection
    ▼
api.post('/visits/check-in', {...})  → axios throws (network error, no `.response`)
    ▼
Screen's catch block detects a network error (not a validation error) →
    syncManager.enqueueAction('post', '/visits/check-in', data, { localId: 'offline-', resolves: 'visit' })
    │  pushes {id, method, url, data, localId, resolves, retryCount:0, timestamp} onto an array
    │  AsyncStorage.setItem('@offline_action_queue', JSON.stringify(queue))
    ▼
UI immediately shows the check-in as done locally (optimistic), with a "pending sync" indicator
    ▼
   ... time passes, rep's phone regains signal ...
    ▼
syncManager's NetInfo listener (startAutoSync, App.js:88) fires: isOnline transitions false → true
    ▼
flushQueue()
    │  for each queued action, in original order:
    │    rewrite any 'offline-...' placeholder id using ids already resolved earlier this pass
    │    if the id it depends on hasn't resolved yet → keep queued, try next flush
    │    else → api.request({method, url, data})
    │      success → record real server id in idMap (if resolves is set), persist remaining queue
    │      network error again → keep queued, retry next time (unbounded — it's not the request's fault)
    │      409 conflict → reconcile via conflictHandler (App.js:97-101) instead of silently dropping
    │      other 4xx → bounded retry, MAX_RETRIES = 8, then discard permanently
```

> **Why bound retries for ordinary 4xx errors but never for network errors:** a network error means "we don't know if this will ever succeed, but it's not this specific request's fault" — worth retrying indefinitely since connectivity will eventually return. A 4xx error (say, a validation failure) means the server actively rejected this exact request — retrying it unchanged will keep failing the same way forever, so after 8 attempts it's discarded rather than silently wedging the whole queue behind a permanently-broken item.

> **Why the `localId` / `resolves` / `idMap` mechanism exists at all:** if a rep checks in for the day while offline, the local UI has no real database id for that attendance record yet — only a temporary placeholder like `"offline-1721300000"`. If they then also check in at a dealer while still offline, that dealer visit references `attendance_id: "offline-1721300000"`, which doesn't exist on the server. When the queue flushes, the day check-in syncs first and returns a real numeric id; `idMap` remembers "offline-1721300000 → 4821", and the dealer check-in action, still waiting in the queue, gets that placeholder rewritten to `4821` before it's sent — preserving the correct relationship between the two records even though neither existed on the server when the rep created them.

PART V

Key File Deep Dives

Each entry below explains the file's role, its most important lines, and what would break if it were removed — reserved for files not already fully traced line-by-line inside a Part IV flow (those are cross-referenced instead of repeated).

5.1 mobile/App.js (415 lines)

Why this file exists: it is the single owner of all cross-screen state — `employee`, `attendance`, `visits`, `selectedDealer`, sync/permission status — and the single place that wires up navigation, the offline-sync lifecycle, and periodic visit monitoring. Every tab screen reads this state via `AppStateContext` (Section 3.6) rather than owning any of it independently.

Imports worth pausing on (lines 1-15)

`AppState` from `react-native` (line 2) is the OS-level foreground/background signal — distinct from this file's own `appStateValue` object (line 290) despite the similar name; it's used at line 157 to re-check location permission when the app returns to the foreground (in case the user revoked it via OS Settings while backgrounded). `setAuthInvalidatedHandler` from `api.js` is a callback registration — `api.js` can't import React or navigate anywhere itself (it's a plain service module), so it calls this handler instead, and `App.js` supplies what "invalidated" actually means for the UI (lines 168-179): reset all state and force navigation back to Login.

The computed `dayStatus` (lines 75-79)

```
const dayStatus = !attendance
  ? 'not_checked_in'
  : attendance.check_out_time
    ? 'day_ended'
    : 'checked_in';
```

This is not stored in `useState` — it's recomputed on every render directly from `attendance`. **Why:** it's fully derivable from existing state, so making it its own piece of state would risk it going stale/out-of-sync with `attendance` if some code path updated one but forgot the other. Deriving it inline guarantees it's always consistent.

What would break if this file disappeared

Every screen would lose its data source — `useAppState()` (Section 3.6) throws an explicit error if called outside `AppStateContext.Provider`, which only `App.js` renders. There would also be no navigation stack at all, since `NavigationContainer` and `Stack.Navigator` are both declared here.

5.2 mobile/screens/LoginScreen.js

Traced fully in Section 4.2. One additional detail worth flagging: the `COLD_START_HINT_DELAY_MS` constant (line 17) and `showColdStartHint` state exist because this backend, per its README-referenced free-tier hosting, can take 30-60 seconds to "wake up" from an idle sleep on the very first request after a period of inactivity (a well-known behavior of free-tier hosts like Render's free web service). Rather than let the rep stare at an ambiguous spinner for a minute wondering if the app is frozen, a hint appears after 4 seconds explaining the likely cause. This is a UX decision responding to a specific infrastructure constraint, not a bug workaround.

5.3 mobile/src/services/api.js

Why this file exists: it is the single shared axios instance for the entire mobile app, so token-attachment and refresh-on-expiry logic is written exactly once instead of duplicated in every screen that makes a request.

The request interceptor (lines 51-60)

Runs before every outgoing request; reads the access token from secure storage and attaches it as an `Authorization: Bearer <token>` header. This is why no individual screen ever manually attaches a token — it happens transparently for every call made through `api`.

The response interceptor and token refresh (lines 63-113)

This is the most intricate piece of logic in the mobile codebase. Walking it precisely: when any request comes back with a 401 (unauthorized) status, and it isn't the login/refresh endpoint itself, and it hasn't already been retried once (`originalRequest._retry` flag), the interceptor assumes the access token expired mid-session and attempts a silent refresh. The `refreshPromise` module-level variable (line 48) is a deliberate concurrency guard: if two requests fail with 401 around the same moment (e.g. a foreground screen fetch and a background sync flush), only the first triggers an actual refresh call — the second awaits the same in-flight promise instead of independently consuming the refresh token a second time. Once refreshed, the original failed request is retried once with the new token (line 98); if the refresh itself fails (refresh token also expired), tokens are cleared and `authInvalidatedHandler` (set by `App.js`) fires to bounce the user to Login.

> **What would break without the `refreshPromise` guard:** two concurrent 401s would each call `/auth/refresh` independently. Since a refresh token, once used, could plausibly be rotated/invalidated by a stricter backend implementation, the second call could fail unpredictably or the two calls could race in ways that leave the app holding a stale token. This codebase's backend doesn't currently rotate refresh tokens on use, so this particular failure mode is muted here — but the guard is correct defensive design regardless, and becomes load-bearing the moment refresh-token rotation is added.

5.4 mobile/src/services/syncManager.js

Traced in full in Section 4.5. Structurally, note the `flushInFlight` pattern (lines 9, 86-93, 191): identical in spirit to `refreshPromise` in `api.js` — it collapses concurrent calls to `flushQueue()` (which can be triggered independently by a NetInfo event, an interval, and a manual retry button) into a single actual flush pass, rather than letting several run in parallel and double-submit queued actions.

5.5 mobile/src/services/location.js

Why it retries up to 3 times (lines 76-103) instead of taking the first GPS reading: the first fix after the GPS radio wakes from an idle/sleep state is frequently the least accurate one, sometimes 50+ meters off. Rather than accept a possibly-bad first reading, the function takes up to three readings (each bounded by a 15-second timeout so one bad attempt can't hang the whole process — `withTimeout`, lines 32-37) and keeps whichever was most accurate, stopping early once a reading is "good enough" (≤20m).

Why reverse geocoding (`getReadableAddress`, lines 127-157) calls the backend instead of a maps API directly from the phone: this keeps the Google Geocoding API key server-side only — it never ships inside the mobile app's bundle, where it could be extracted and abused by anyone who decompiles the APK/IPA.

5.6 backend/src/index.js

Why it refuses to boot without `JWT_SECRET` (lines 33-38): if this weren't checked at startup, the first login attempt would crash with a confusing internal error from deep inside `jwt.sign` instead of a clear message at the moment it's most actionable (server startup, in the terminal the developer is watching). It further refuses to start in production with the exact placeholder secret shipped in `.env.example` — without that check, someone could accidentally deploy to production having forgotten to set a real secret, which would make every issued token forgeable by anyone who has read the (public) example file.

Why CORS logic is dynamic (lines 54-107) instead of a fixed list: in local development, the machine's LAN IP changes on every Wi-Fi reconnect/DHCP renewal, and Expo Go's own networking can send an `exp://`-scheme origin rather than `http(s)://`. The code explicitly allows any localhost or private-LAN (RFC 1918) origin during development, while requiring an explicit `ALLOWED_ORIGINS` list in production (enforced by throwing at startup if missing, mirroring the JWT_SECRET check) — so the permissive dev-only fallback can never accidentally ship to a real deployment.

Route mounting order (lines 138-156) is itself meaningful — Express matches middleware/routes in the order they're registered. Public auth routes are mounted first with no auth middleware; every other route is mounted with `requireAuth` (and some additionally with `requireRole('manager')`) directly in the mounting call, meaning the rejection happens before the request ever reaches that router's own code.

5.7 backend/src/routes/auth.routes.js

Traced in Section 4.2. One detail to add: `signAccessToken` and `signRefreshToken` (lines 15-29) both embed `employee.id` as the JWT's `sub` (subject) claim — a JWT convention for "who this token is about." The refresh token additionally embeds `{ type: 'refresh'`

`}`, which `/auth/refresh` explicitly checks (line 95) — this stops someone from taking an access token and attempting to use it directly against the refresh endpoint, since it would fail the `type !== 'refresh'` check.

5.8 backend/src/middleware/auth.middleware.js

Why `isEmployeeActive` exists and is cached (lines 15-27): a JWT, once issued, is self-contained and stateless — the server can verify it's authentic without a database lookup at all. But that means deactivating an employee (e.g. after termination) wouldn't stop their still-unexpired access token from working for up to 8 more hours, purely from JWT verification alone. This function adds back exactly the one piece of state that must be checked live: whether the employee is still active — cached for 30 seconds per employee id so this doesn't become a full database round trip on literally every request, while still capping the "still works after deactivation" window to well under a minute rather than up to 8 hours.

requireRole (lines 58-68) is a higher-order function — it takes a role string and *returns* a middleware function, which is why it's called as `requireRole('manager')` at the mount site rather than passed as a bare reference — the call configures which role it checks before Express ever invokes it per-request.

5.9 backend/src/routes/attendance.routes.js

Traced in Section 4.3 (check-in) and Section 4.5 conceptually (queued check-outs reconciling via 409). The `check-out` handler's 409 branch (lines 107-115) is the server-side half of the reconciliation mechanism described in 4.5 — it hands back the already-recorded checkout time rather than erroring generically, specifically so a retried offline-queued action can adopt that as truth instead of the sync manager treating it as an unrecoverable failure.

5.10 backend/src/routes/visits.routes.js

Traced in Section 4.4 for check-in; check-out (lines 174-297) adds one refinement worth understanding: if the checkout GPS is outside the dealer's radius, it isn't automatically flagged as an exception — first it checks whether the checkout point is within a small tolerance (`MATCH_TOLERANCE_M`, 20m) of where the rep originally checked in (lines 241-245). **Why:** normal GPS drift at a stationary location (the rep never moved, but the phone's fix wandered slightly) shouldn't require a written justification the way a genuine out-of-radius checkout should — this distinguishes "GPS noise at the same spot" from "the rep actually appears to have left."

The `/interrupt` and `/location-check` endpoints (lines 307-455) back the "Random Location Verification" feature traced conceptually in `App.js`'s `startVisitMonitoring` effect (Section 5.1) — the phone pings the backend every ~10 minutes during an open visit, and the backend accumulates a non-resetting count of out-of-radius pings, tripping a "time to log out" alert (both to the rep's own device and flagged for manager review) once that count reaches 2, even if the two breaches weren't consecutive.

5.11 backend/src/utils/haversine.js and businessDay.js

haversine.js implements the Haversine formula — the standard way to compute great-circle distance between two latitude/longitude points on a sphere, accounting for the Earth's curvature (a naive Pythagorean distance on raw lat/lng degrees would be wrong, and increasingly wrong the further apart or the higher the latitude of the two points). It is deliberately a small, dependency-free, directly unit-tested module used identically in three places (its own header comment enumerates them) — check-in distance-from-previous, check-out radius validation, and total daily distance accumulation — so the actual distance math exists in exactly one place rather than being reimplemented slightly differently three times.

businessDay.js centralizes the "the workday rolls over at 5am, not midnight" rule (Section 4.3) into two exported SQL-fragment-generating functions rather than a JS function — because the logic needs to run *inside SQL queries* (both for the `ON CONFLICT` guard and the `WHERE` clause filtering "today's" attendance), not just in application code. `DAY_BOUNDARY_HOUR` is read once at process startup (module-load time) — the file's own comment flags that changing this environment variable requires restarting the server to take effect, since the constant is computed once and reused for the life of the process.

5.12 backend/src/db/schema.sql

Five tables, in dependency order: `employees` (no foreign keys — the root), `dealers` (also root-level), `attendance` (references `employees`), `client_visits` (references both `attendance` and `dealers`), and `exception_log` (references all three transitively, directly referencing `employees`, `dealers`, and `client_visits`). A sixth table, `notes`, is unrelated to the geofencing domain — free-

form notepad entries per employee, with a 100-character minimum enforced by a `CHECK` constraint at the database level (line 187), not just in the mobile app's form validation, specifically so the rule holds regardless of which client ever writes to that table.

The file is written as a sequence of `CREATE TABLE IF NOT EXISTS` statements followed by `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` statements layered on afterward — this is the "plain SQL, re-runnable migration" strategy the backend README describes explicitly choosing over a dedicated migrations folder with numbered incremental files (the more common ORM-tooling convention). **Why this works here:** every statement is idempotent (safe to run again with no effect if already applied), so `migrate.js` can simply re-execute the entire file against any environment — a fresh database or one that already has some of these columns — without needing a migration-tracking table to know what's already been applied.

The unique index enabling the atomic day-check-in guard (Section 4.3) is worth re-reading directly in context:

```
CREATE UNIQUE INDEX IF NOT EXISTS idx_attendance_employee_business_date
  ON attendance (employee_id, business_date) WHERE business_date IS NOT NULL;
```

This is a *partial* unique index (the `WHERE` clause) — it only enforces uniqueness for rows where `business_date` is actually set, which matters because the column was added later via `ALTER TABLE` (line 69) to a table that could already contain older rows with no `business_date` value; without the partial condition, those old null-valued rows could conflict with each other under a naive unique constraint (most databases, including Postgres, treat NULLs as "not equal to each other" for uniqueness purposes by default — but the explicit `WHERE` clause here removes any ambiguity and intent-documents it).

PART VI

Closing

6.1 Common mistakes across this codebase's domain

- Trusting client-computed booleans (e.g. "am I inside the radius") instead of re-deriving them server-side from raw inputs — this is the single most repeated defensive pattern across `visits.routes.js`.
- Forgetting that an unhandled promise rejection never retries itself (Section 3.3) — leads to silently stuck loading states or silently lost offline data.
- Mutating React state arrays/objects in place instead of using spread (Section 3.2) — causes React to skip re-renders it should have performed.
- Building SQL queries via string concatenation of user input instead of parameterized placeholders (Section 3.9) — the classic SQL injection vector.
- Using calendar midnight instead of a configurable business-day boundary for anything attendance-related (Section 4.3) — breaks for any shift crossing midnight.
- Missing the `useEffect` cleanup function when starting a subscription/listener/timer (Section 3.6) — leaks the old one on every re-run or unmount.

6.2 Interview questions bank

- Why does this backend use raw SQL instead of an ORM, and what would justify switching later?
- Walk through what happens, end to end, when an access token expires mid-session on the mobile app.
- Why are there two JWTs (access + refresh) instead of one, and what does each one's lifetime protect against?
- Explain the race condition `ON CONFLICT ... DO NOTHING` prevents, and why a check-then-insert approach is insufficient.
- Why must GPS-based geofencing decisions be re-validated server-side even when the client already checked them?
- What problem does the offline action queue's `localId` / `idMap` rewriting solve, concretely?
- What is the difference between an arrow function with and without curly braces, with respect to implicit return?
- Why is a partial unique index (with a WHERE clause) sometimes necessary instead of a plain unique constraint?

6.3 Exercises and coding challenges

1. Trace, on paper, what HTTP requests fire and in what order if a rep opens the app fully offline, checks in for the day, checks in at a dealer, then regains signal 20 minutes later. Name every file involved at each step.
2. Write a small Node script (no need to run it against the real DB) that calls `haversineKm` with two coordinates 100 meters apart and confirms the returned distance is close to 0.1 km.
3. Identify: which single line in `schema.sql` is responsible for preventing two attendance rows for the same employee on the same business day, and which single line in `attendance.routes.js` relies on it?
4. Modify (on paper) `syncManager.js`'s retry logic to also give up early on a 401 response instead of only on generic 4xx after 8 retries — explain why this would actually be a meaningful improvement, given what a 401 means for an offline-queued action.

6.4 What to read next / study roadmap

This document intentionally does not cover every file in the repository — it covers the architectural spine and the highest-traffic flows. Suggested next passes, each following the same "read every line, ask why, verify understanding" method used throughout this document:

1. `mobile/src/services/visitMonitor.js` and the "Random Location Verification" feature end to end, including the manager-facing side in `dashboard.routes.js`.
2. The web dashboard's heatmap (`web/src/components/heatmap/`) — how deck.gl consumes visit data to render density.
3. `backend/src/routes/dealers.routes.js` and `employees.routes.js` — the manager-side CRUD operations not covered here.
4. The test suites in `backend/tests/` — reading tests is often the fastest way to understand a route's edge cases, since each test name states an expected behavior explicitly.

5. React Navigation's stack vs. tab navigator model in more depth — this document only traced how they're used, not their full API surface.

— End of document —